

CS162
Operating Systems and
Systems Programming
Lecture 9

Monitors (Continued)

Professor Ion Stoica & Matei Zaharia

<https://cs162.org/>

Where are we going with synchronization?

Programs	Shared Programs
Higher-level API	Locks Semaphores Monitors Send/Receive
Hardware	Load/Store Disable Interrupts Test&Set Compare&Swap

Implement various higher-level synchronization primitives using atomic operations

Recall: Semaphores

A Semaphore has a **non-negative integer value** and supports the following operations:

- Set starting value when you initialize
- **Down()** or **P()**: an atomic operation that waits for semaphore to become positive, then decrements it by 1
 - » Think of this as the wait() operation
- **Up()** or **V()**: an atomic operation that increments the semaphore by 1, waking up a waiting P, if any
 - » Think of this as the signal() operation

Two Uses of Semaphores

Mutual Exclusion (initial value = 1)

Also called “Binary Semaphore” or “mutex”.

Can be used for mutual exclusion, just like a lock:

```
semaP(&mysem);  
    // Critical section goes here  
semaV(&mysem);
```

Two Uses of Semaphores

Scheduling Constraints (initial value = 0)

Allow thread 1 to wait for a signal from thread 2

- thread 2 **schedules** thread 1 when a given **event** occurs

Example: suppose you had to implement ThreadJoin which must wait for thread to terminate:

```
Initial value of semaphore = 0
ThreadJoin {
    semaP(&mysem);
}
ThreadFinish {
    semaV(&mysem);
}
```

Recall: Monitors

Use *locks* for mutual exclusion and *condition variables* for scheduling constraints

Monitor: a **lock** and zero or more **condition variables** for managing concurrent access to shared data

Recall: Wait & Signal Pattern

```
...  
acquire(&buf_lock)  
...  
cond_signal(&buf_CV);  
...  
release(&buf_lock));
```

```
acquire(&buf_lock);  
...  
while (isEmpty(&queue)) {  
    cond_wait(&buf_CV,&buf_lock);  
}  
...  
lock.Release();
```

Recall: Hoare Semantics

Thread A

```
...  
acquire(&buf_lock)  
...  
cond_signal(&buf_CV);  
...  
release(&buf_lock));
```

Thread B

```
acquire(&buf_lock);  
...  
if (isEmpty(&queue)) {  
    cond_wait(&buf_CV, &buf_lock);  
}  
...  
lock.Release();
```

1. When call signal, hand over buf_lock to thread B.
2. Thread B gets immediately scheduled (nothing can run in between).
3. Thread B eventually releases lock.

Recall: Mesa Semantics

Thread A

```
...  
acquire(&buf_lock)  
...  
cond_signal(&buf_CV);  
...  
release(&buf_lock));
```

Thread B

```
acquire(&buf_lock);  
...  
while (isEmpty(&queue)) {  
    cond_wait(&buf_CV,&buf_lock);  
}  
...  
lock.Release();
```

1. When call signal, keep lock. Place Thread B on READY queue (no special priority)
2. Thread A eventually releases buf_lock.
3. Thread B eventually gets scheduled and acquires buf_lock. Thread C may have run in between.
4. Thread B eventually releases buf_lock.

Basic Structure of *Mesa* Monitor Program

Monitors represent the synchronization logic of the program

- Wait if necessary
- Signal when change something so any waiting threads can proceed

```
lock
while (need to wait) {
    condvar.wait();
}
unlock
```



Check and/or update
state variables
Wait if necessary

```
do something so no need to wait
lock

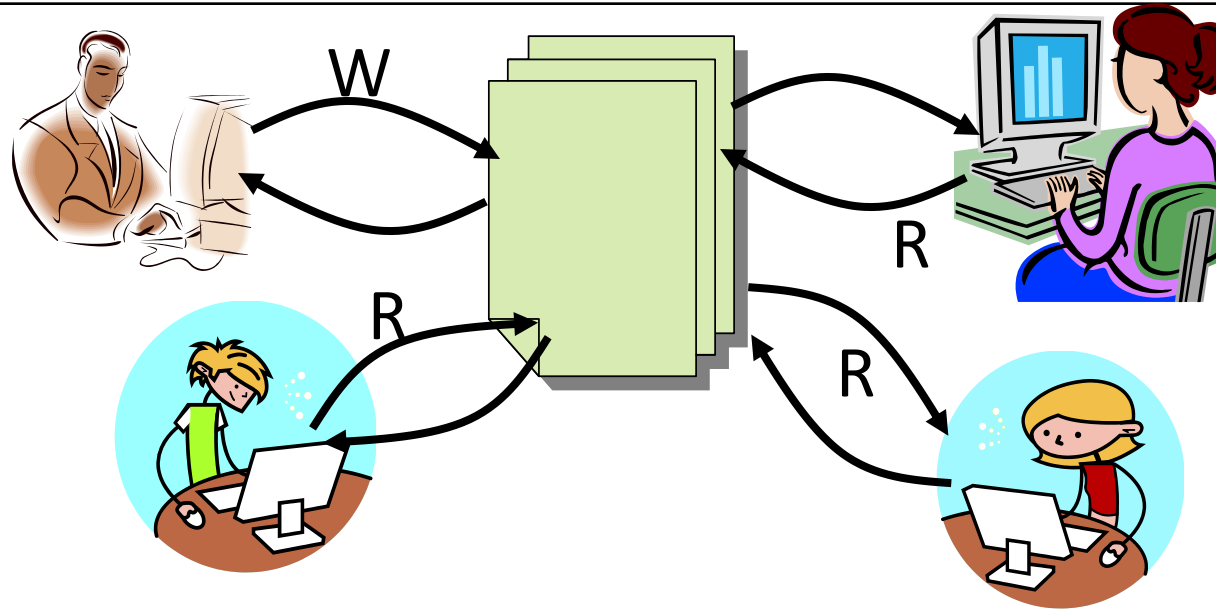
condvar.signal();

unlock
```



Check and/or update
state variables

The Readers/Writers Problem



Motivation: consider a shared database

- Two classes of users:
 - » Readers – never modify database
 - » Writers – read and modify database
- Is using a single lock on the whole database sufficient?
 - » Like to have many readers at the same time
 - » Only one writer at a time

Basic Readers/Writers Solution

Correctness Constraints:

- Readers can access database when no writers
- Writers can access database when no readers or writers
- Only one thread manipulates state variables at a time

Basic structure of a solution:

- Reader()
 - Wait until no writers
 - Access database
 - Check out – wake up a waiting writer
- Writer()
 - Wait until no active readers or writers
 - Access database
 - Check out – wake up waiting readers or writer

Basic Readers/Writers Solution

State variables (Protected by a lock called “lock”):

- » int AR: Number of active readers; initially = 0
- » int WR: Number of waiting readers; initially = 0
- » int AW: Number of active writers; initially = 0
- » int WW: Number of waiting writers; initially = 0
- » Condition okToRead
- » Condition okToWrite

Code for a Reader

```
Reader() {
    // First check self into system
    acquire(&lock);
    while ((AW + WW) > 0) {    // Is it safe to read?
        WR++;                // No. Writers exist
        cond_wait(&okToRead,&lock); // Sleep on cond var
        WR--;                // No longer waiting
    }
    AR++;                    // Now we are active!
    release(&lock);

    // Perform actual read-only access
    AccessDatabase(ReadOnly);

    // Now, check out of system
    acquire(&lock);
    AR--;                    // No longer active
    if (AR == 0 && WW > 0)    // No other active readers
        cond_signal(&okToWrite); // Wake up one writer
    release(&lock);
}
```

Code for a Writer

```
Writer() {
    // First check self into system
    acquire(&lock);
    while ((AW + AR) > 0) {    // Is it safe to write?
        WW++;                // No. Active users exist
        cond_wait(&okToWrite,&lock);    // Sleep on cond var
        WW--;                // No longer waiting
    }
    AW++;                    // Now we are active!
    release(&lock);
    // Perform actual read/write access
    AccessDatabase(ReadWrite);
    // Now, check out of system
    acquire(&lock);
    AW--;                    // No longer active
    if (WW > 0){              // Give priority to writers
        cond_signal(&okToWrite); // Wake up one writer
    } else if (WR > 0) {      // Otherwise, wake reader
        cond_broadcast(&okToRead); // Wake all readers
    }
    release(&lock);
}
```

Simulation of Readers/Writers Solution

Use an example to simulate the solution

Consider the following sequence of operators:

– R1, R2, W1, R3

Initially: $AR = 0$, $WR = 0$, $AW = 0$, $WW = 0$

Simulation of Readers/Writers Solution

R1 comes along (no waiting threads)

AR = 0, WR = 0, AW = 0, WW = 0

```
Reader() {
    acquire(&lock)
    while ((AW + WW) > 0) { // Is it safe to read?
        WR++;              // No. Writers exist
        cond_wait(&okToRead, &lock); // Sleep on cond var
        WR--;              // No longer waiting
    }
    AR++;                  // Now we are active!
    release(&lock);

    AccessDBase(ReadOnly);

    acquire(&lock);
    AR--;
    if (AR == 0 && WW > 0)
        cond_signal(&okToWrite);
    release(&lock);
}
```

Simulation of Readers/Writers Solution

R1 comes along (no waiting threads)

$AR = 0, WR = 0, AW = 0, WW = 0$

```
Reader() {
    acquire(&lock);
    while ((AW + WW) > 0) { // Is it safe to read?
        WR++;              // No. Writers exist
        cond_wait(&okToRead, &lock); // Sleep on cond var
        WR--;              // No longer waiting
    }
    AR++;                  // Now we are active!
    release(&lock);

    AccessDBase(ReadOnly);

    acquire(&lock);
    AR--;
    if (AR == 0 && WW > 0)
        cond_signal(&okToWrite);
    release(&lock);
}
```

Simulation of Readers/Writers Solution

R1 comes along (no waiting threads)

AR = 1, WR = 0, AW = 0, WW = 0

```
Reader() {
    acquire(&lock);
    while ((AW + WW) > 0) { // Is it safe to read?
        WR++;              // No. Writers exist
        cond_wait(&okToRead, &lock); // Sleep on cond var
        WR--;              // No longer waiting
    }
    AR++;                  // Now we are active!
    release(&lock);

    AccessDBase(ReadOnly);

    acquire(&lock);
    AR--;
    if (AR == 0 && WW > 0)
        cond_signal(&okToWrite);
    release(&lock);
}
```

Simulation of Readers/Writers Solution

R1 comes along (no waiting threads)

AR = 1, WR = 0, AW = 0, WW = 0

```
Reader() {
    acquire(&lock);
    while ((AW + WW) > 0) { // Is it safe to read?
        WR++;              // No. Writers exist
        cond_wait(&okToRead, &lock); // Sleep on cond var
        WR--;              // No longer waiting
    }
    AR++;                  // Now we are active!
    release(&lock);
```

AccessDBase(ReadOnly);

```
    acquire(&lock);
    AR--;
    if (AR == 0 && WW > 0)
        cond_signal(&okToWrite);
    release(&lock);
}
```

Simulation of Readers/Writers Solution

R1 accessing dbase (no other threads)

AR = 1, WR = 0, AW = 0, WW = 0

```
Reader() {
    acquire(&lock);
    while ((AW + WW) > 0) { // Is it safe to read?
        WR++;              // No. Writers exist
        cond_wait(&okToRead, &lock); // Sleep on cond var
        WR--;              // No longer waiting
    }
    AR++;                  // Now we are active!
    release(&lock);
}
```

AccessDBase(ReadOnly);

```
    acquire(&lock);
    AR--;
    if (AR == 0 && WW > 0)
        cond_signal(&okToWrite);
    release(&lock);
}
```

Simulation of Readers/Writers Solution

R2 comes along (R1 accessing dbase)

AR = 1, WR = 0, AW = 0, WW = 0

```
Reader() {
    acquire(&lock);
    while ((AW + WW) > 0) { // Is it safe to read?
        WR++;              // No. Writers exist
        cond_wait(&okToRead, &lock); // Sleep on cond var
        WR--;              // No longer waiting
    }
    AR++;                  // Now we are active!
    release(&lock);

    AccessDBase(ReadOnly);

    acquire(&lock);
    AR--;
    if (AR == 0 && WW > 0)
        cond_signal(&okToWrite);
    release(&lock);
}
```

Simulation of Readers/Writers Solution

R2 comes along (R1 accessing dbase)

$AR = 1, WR = 0, AW = 0, WW = 0$

```
Reader() {
    acquire(&lock);
    while ((AW + WW) > 0) { // Is it safe to read?
        WR++;              // No. Writers exist
        cond_wait(&okToRead, &lock); // Sleep on cond var
        WR--;              // No longer waiting
    }
    AR++;                  // Now we are active!
    release(&lock);

    AccessDBase(ReadOnly);

    acquire(&lock);
    AR--;
    if (AR == 0 && WW > 0)
        cond_signal(&okToWrite);
    release(&lock);
}
```

Simulation of Readers/Writers Solution

R2 comes along (R1 accessing dbase)

AR = 2, WR = 0, AW = 0, WW = 0

```
Reader() {
    acquire(&lock);
    while ((AW + WW) > 0) { // Is it safe to read?
        WR++;              // No. Writers exist
        cond_wait(&okToRead, &lock); // Sleep on cond var
        WR--;              // No longer waiting
    }
    AR++;                  // Now we are active!
    release(&lock);

    AccessDBase(ReadOnly);

    acquire(&lock);
    AR--;
    if (AR == 0 && WW > 0)
        cond_signal(&okToWrite);
    release(&lock);
}
```


Simulation of Readers/Writers Solution

R2 comes along (R1 accessing dbase)

AR = 2, WR = 0, AW = 0, WW = 0

```
Reader() {
    acquire(&lock);
    while ((AW + WW) > 0) { // Is it safe to read?
        WR++;              // No. Writers exist
        cond_wait(&okToRead, &lock); // Sleep on cond var
        WR--;              // No longer waiting
    }
    AR++;                  // Now we are active!
    release(&lock);
```

AccessDBase(ReadOnly);

```
    acquire(&lock);
    AR--;
    if (AR == 0 && WW > 0)
        cond_signal(&okToWrite);
    release(&lock);
}
```

Simulation of Readers/Writers Solution

R1 and R2 accessing dbase

AR = 2, WR = 0, AW = 0, WW = 0

```
Reader() {
    acquire(&lock);
    while ((AW + WW) > 0) { // Is it safe to read?
        WR++;              // No. Writers exist
        cond_wait(&okToRead, &lock); // Sleep on cond var
        WR--;              // No longer waiting
    }
    AR++;                  // Now we are active!
    release(&lock);
}
```

AccessDBase(ReadOnly);

```
    acquire(&lock);
    AR--;
    if (AR == 0 && WW > 0)
```

Assume readers take a while to access database
Situation: Locks released, only AR is non-zero

Simulation of Readers/Writers Solution

W1 comes along (R1 and R2 are still accessing dbase)

$AR = 2, WR = 0, AW = 0, WW = 0$

```
Writer() {
    acquire(&lock);
    while ((AW + AR) > 0) {           // Is it safe to write?
        WW++;                         // No, Active users exist
        cond_wait(&okToWrite, &lock); // Sleep on cond var
        WW--;                         // No longer waiting
    }
    AW++;
    release(&lock);
}
```

AccessDBase(ReadWrite);

```
    acquire(&lock);
    AW--;
    if (WW > 0) {
        cond_signal(&okToWrite);
    } else if (WR > 0) {
        cond_broadcast(&okToRead);
    }
    release(&lock);
}
```

Simulation of Readers/Writers Solution

W1 comes along (R1 and R2 are still accessing dbase)

$AR = 2, WR = 0, AW = 0, WW = 0$

```
Writer() {
    acquire(&lock);
    while ((AW + AR) > 0) { // Is it safe to write?
        WW++; // No, Active users exist
        cond_wait(&okToWrite, &lock); // Sleep on cond var
        WW--; // No longer waiting
    }
    AW++;
    release(&lock);
}
```

AccessDBase(ReadWrite);

```
    acquire(&lock);
    AW--;
    if (WW > 0) {
        cond_signal(&okToWrite);
    } else if (WR > 0) {
        cond_broadcast(&okToRead);
    }
    release(&lock);
}
```

Simulation of Readers/Writers Solution

- W1 comes along (R1 and R2 are still accessing dbase)
- $AR = 2$, $WR = 0$, $AW = 0$, $WW = 1$

```
Writer() {
    acquire(&lock);
    while ((AW + AR) > 0) {           // Is it safe to write?
        WW++;                         // No. Active users exist
        cond_wait(&okToWrite, &lock); // Sleep on cond var
        WW--;                         // No longer waiting
    }
    AW++;
    release(&lock);
}
```

AccessDBase(ReadWrite);

```
    acquire(&lock);
    AW--;
    if (WW > 0) {
        cond_signal(&okToWrite);
    } else if (WR > 0) {
        cond_broadcast(&okToRead);
    }
    release(&lock);
}
```

Simulation of Readers/Writers Solution

R3 comes along (R1 and R2 accessing dbase, W1 waiting)

AR = 2, WR = 0, AW = 0, WW = 1

```
Reader() {  
    acquire(&lock);  
    while ((AW + WW) > 0) { // Is it safe to read?  
        WR++; // No. Writers exist  
        cond_wait(&okToRead, &lock); // Sleep on cond var  
        WR--; // No longer waiting  
    }  
    AR++; // Now we are active!  
    release(&lock);  
  
    AccessDBase(ReadOnly);  
  
    acquire(&lock);  
    AR--;  
    if (AR == 0 && WW > 0)  
        cond_signal(&okToWrite);  
    release(&lock);  
}
```

Simulation of Readers/Writers Solution

R3 comes along (R1 and R2 accessing dbase, W1 waiting)

$AR = 2$, $WR = 0$, $AW = 0$, $WW = 1$

```
Reader() {
    acquire(&lock);
    while ((AW + WW) > 0) { // Is it safe to read?
        WR++;              // No. Writers exist
        cond_wait(&okToRead, &lock); // Sleep on cond var
        WR--;              // No longer waiting
    }
    AR++;                  // Now we are active!
    release(&lock);

    AccessDBase(ReadOnly);

    acquire(&lock);
    AR--;
    if (AR == 0 && WW > 0)
        cond_signal(&okToWrite);
    release(&lock);
}
```

Simulation of Readers/Writers Solution

R3 comes along (R1 and R2 accessing dbase, W1 waiting)

AR = 2, WR = 1, AW = 0, WW = 1

```
Reader() {
    acquire(&lock);
    while ((AW + WW) > 0) { // Is it safe to read?
        WR++; // No. Writers exist
        cond_wait(&okToRead, &lock); // Sleep on cond var
        WR--; // No longer waiting
    }
    AR++; // Now we are active!
    lock.release();

    AccessDBase(ReadOnly);

    acquire(&lock);
    AR--;
    if (AR == 0 && WW > 0)
        cond_signal(&okToWrite);
    release(&lock);
}
```


Simulation of Readers/Writers Solution

R3 comes along (R1, R2 accessing dbase, W1 waiting)

AR = 2, WR = 1, AW = 0, WW = 1

```
Reader() {
    acquire(&lock);
    while ((AW + WW) > 0) { // Is it safe to read?
        WR++;              // No. Writers exist
        cond_wait(&okToRead, &lock); // Sleep on cond var
        WR--;              // No longer waiting
    }
    AR++;                  // Now we are active!
    release(&lock);
}
```

AccessDBase(ReadOnly);

```
    acquire(&lock);
    AR--;
    if (AR == 0 && WW > 0)
        cond_signal(&okToWrite);
    release(&lock);
}
```

Simulation of Readers/Writers Solution

R1 and R2 accessing dbase, W1 and R3 waiting

$AR = 2, WR = 1, AW = 0, WW = 1$

```
Reader() {
    acquire(&lock);
    while ((AW + WW) > 0) { // Is it safe to read?
        WR++;              // No. Writers exist
        cond_wait(&okToRead, &lock); // Sleep on cond var
        WR--;              // No longer waiting
    }
    AR++;                  // Now we are active!
    release(&lock);
}
```

AccessDBase(ReadOnly);

```
    acquire(&lock);
    AR--;
    if (AR == 0 && WW > 0)
```

Status:

- R1 and R2 still reading
- W1 and R3 waiting on okToWrite and okToRead, respectively

Simulation of Readers/Writers Solution

R2 finishes (R1 accessing dbase, W1 and R3 waiting)

AR = 2, WR = 1, AW = 0, WW = 1

```
Reader() {
    acquire(&lock);
    while ((AW + WW) > 0) { // Is it safe to read?
        WR++;              // No. Writers exist
        cond_wait(&okToRead, &lock); // Sleep on cond var
        WR--;              // No longer waiting
    }
    AR++;                  // Now we are active!
    release(&lock);
}
```

AccessDBase(ReadOnly);

```
    acquire(&lock);
    AR--;
    if (AR == 0 && WW > 0)
        cond_signal(&okToWrite);
    release(&lock);
}
```

Simulation of Readers/Writers Solution

R2 finishes (R1 accessing dbase, W1 and R3 waiting)

AR = 1, WR = 1, AW = 0, WW = 1

```
Reader() {
    acquire(&lock);
    while ((AW + WW) > 0) { // Is it safe to read?
        WR++;              // No. Writers exist
        cond_wait(&okToRead, &lock); // Sleep on cond var
        WR--;              // No longer waiting
    }
    AR++;                  // Now we are active!
    release(&lock);

    AccessDBase(ReadOnly);

    acquire(&lock);
    AR--;
    if (AR == 0 && WW > 0)
        cond_signal(&okToWrite);
    release(&lock);
}
```

Simulation of Readers/Writers Solution

R2 finishes (R1 accessing dbase, W1 and R3 waiting)

AR = 1, WR = 1, AW = 0, WW = 1

```
Reader() {
    acquire(&lock);
    while ((AW + WW) > 0) { // Is it safe to read?
        WR++;              // No. Writers exist
        cond_wait(&okToRead, &lock); // Sleep on cond var
        WR--;              // No longer waiting
    }
    AR++;                  // Now we are active!
    release(&lock);

    AccessDBase(ReadOnly);

    acquire(&lock);
    AR--;
    if (AR == 0 && WW > 0)
        cond_signal(&okToWrite);
    release(&lock);
}
```

Simulation of Readers/Writers Solution

R2 finishes (R1 accessing dbase, W1 and R3 waiting)

$AR = 1, WR = 1, AW = 0, WW = 1$

```
Reader() {
    acquire(&lock);
    while ((AW + WW) > 0) { // Is it safe to read?
        WR++;               // No. Writers exist
        cond_wait(&okToRead, &lock); // Sleep on cond var
        WR--;               // No longer waiting
    }
    AR++;                   // Now we are active!
    release(&lock);

    AccessDBase(ReadOnly);

    acquire(&lock);
    AR--;
    if (AR == 0 && WW > 0)
        cond_signal(&okToWrite);
    release(&lock);
}
```

Simulation of Readers/Writers Solution

R1 finishes (W1 and R3 waiting)

AR = 1, WR = 1, AW = 0, WW = 1

```
Reader() {
    acquire(&lock);
    while ((AW + WW) > 0) { // Is it safe to read?
        WR++;              // No. Writers exist
        cond_wait(&okToRead, &lock); // Sleep on cond var
        WR--;              // No longer waiting
    }
    AR++;                  // Now we are active!
    release(&lock);
}
```

AccessDBase(ReadOnly);

```
    acquire(&lock);
    AR--;
    if (AR == 0 && WW > 0)
        cond_signal(&okToWrite);
    release(&lock);
}
```

Simulation of Readers/Writers Solution

R1 finishes (W1, R3 waiting)

AR = 0, WR = 1, AW = 0, WW = 1

```
Reader() {
    acquire(&lock);
    while ((AW + WW) > 0) { // Is it safe to read?
        WR++;              // No. Writers exist
        cond_wait(&okToRead, &lock); // Sleep on cond var
        WR--;              // No longer waiting
    }
    AR++;                  // Now we are active!
    release(&lock);

    AccessDBase(ReadOnly);

    acquire(&lock);
    AR--;
    if (AR == 0 && WW > 0)
        cond_signal(&okToWrite);
    release(&lock);
}
```


Simulation of Readers/Writers Solution

R1 finishes (W1, R3 waiting)

AR = 0, WR = 1, AW = 0, WW = 1

```
Reader() {
    acquire(&lock);
    while ((AW + WW) > 0) { // Is it safe to read?
        WR++;              // No. Writers exist
        cond_wait(&okToRead, &lock); // Sleep on cond var
        WR--;              // No longer waiting
    }
    AR++;                  // Now we are active!
    release(&lock);

    AccessDBase(ReadOnly);

    acquire(&lock);
    AR--;
    if (AR == 0 && WW > 0)
        cond_signal(&okToWrite);
    release(&lock);
}
```

Simulation of Readers/Writers Solution

R1 signals a writer (W1 and R3 waiting)

$AR = 0, WR = 1, AW = 0, WW = 1$

```
Reader() {
    acquire(&lock);
    while ((AW + WW) > 0) { // Is it safe to read?
        WR++;              // No. Writers exist
        cond_wait(&okToRead, &lock); // Sleep on cond var
        WR--;              // No longer waiting
    }
    AR++;                  // Now we are active!
    release(&lock);

    AccessDBase(ReadOnly);

    acquire(&lock);
    AR--;
    if (AR == 0 && WW > 0)
        cond_signal(&okToWrite);
    release(&lock);
}
```

Simulation of Readers/Writers Solution

W1 gets signal (R3 still waiting)

AR = 0, WR = 1, AW = 0, WW = 1

```
Writer() {
    acquire(&lock);
    while ((AW + AR) > 0) {          // Is it safe to write?
        WW++;                       // No, Active users exist
        cond_wait(&okToWrite, &lock); // Sleep on cond var
        WW--;                       // No longer waiting
    }
    AW++;
    release(&lock);
}
```

AccessDBase(ReadWrite);

```
    acquire(&lock);
    AW--;
    if (WW > 0) {
        cond_signal(&okToWrite);
    } else if (WR > 0) {
        cond_broadcast(&okToRead);
    }
    release(&lock);
}
```

Simulation of Readers/Writers Solution

W1 gets signal (R3 still waiting)

AR = 0, WR = 1, AW = 0, WW = 0

```
Writer() {
    acquire(&lock);
    while ((AW + AR) > 0) {           // Is it safe to write?
        WW++;                         // No, Active users exist
        cond_wait(&okToWrite, &lock); // Sleep on cond var
        WW--;                         // No longer waiting
    }
    AW++;
    release(&lock);
}
```

AccessDBase(ReadWrite);

```
    acquire(&lock);
    AW--;
    if (WW > 0) {
        cond_signal(&okToWrite);
    } else if (WR > 0) {
        cond_broadcast(&okToRead);
    }
    release(&lock);
}
```

Simulation of Readers/Writers Solution

W1 gets signal (R3 still waiting)

AR = 0, WR = 1, AW = 1, WW = 0

```
Writer() {
    acquire(&lock);
    while ((AW + AR) > 0) {           // Is it safe to write?
        WW++;                         // No, Active users exist
        cond_wait(&okToWrite, &lock); // Sleep on cond var
        WW--;                         // No longer waiting
    }
    AW++;
    release(&lock);
}
```

AccessDBase(ReadWrite);

```
    acquire(&lock);
    AW--;
    if (WW > 0) {
        cond_signal(&okToWrite);
    } else if (WR > 0) {
        cond_broadcast(&okToRead);
    }
    release(&lock);
}
```

Simulation of Readers/Writers Solution

W1 accessing dbase (R3 still waiting)

AR = 0, WR = 1, AW = **1**, WW = 0

```
Writer() {
    acquire(&lock);
    while ((AW + AR) > 0) {           // Is it safe to write?
        WW++;                         // No, Active users exist
        cond_wait(&okToWrite, &lock); // Sleep on cond var
        WW--;                         // No longer waiting
    }
    AW++;
    release(&lock);
}
```

AccessDBase(ReadWrite);

```
    acquire(&lock);
    AW--;
    if (WW > 0) {
        cond_signal(&okToWrite);
    } else if (WR > 0) {
        cond_broadcast(&okToRead);
    }
    release(&lock);
}
```

Simulation of Readers/Writers Solution

W1 finishes (R3 still waiting)

AR = 0, WR = 1, AW = 1, WW = 0

```
Writer() {
    acquire(&lock);
    while ((AW + AR) > 0) {           // Is it safe to write?
        WW++;                         // No, Active users exist
        cond_wait(&okToWrite, &lock); // Sleep on cond var
        WW--;                         // No longer waiting
    }
    AW++;
    release(&lock);
}
```

AccessDBase(ReadWrite);

```
acquire(&lock);
AW--;
if (WW > 0) {
    cond_signal(&okToWrite);
} else if (WR > 0) {
    cond_broadcast(&okToRead);
}
release(&lock);
}
```

Simulation of Readers/Writers Solution

W1 finishes (R3 still waiting)

AR = 0, WR = 1, AW = 0, WW = 0

```
Writer() {
    acquire(&lock);
    while ((AW + AR) > 0) {           // Is it safe to write?
        WW++;                         // No, Active users exist
        cond_wait(&okToWrite, &lock); // Sleep on cond var
        WW--;                         // No longer waiting
    }
    AW++;
    release(&lock);
}
```

AccessDBase(ReadWrite);

```
    acquire(&lock);
    AW--;
    if (WW > 0) {
        cond_signal(&okToWrite);
    } else if (WR > 0) {
        cond_broadcast(&okToRead);
    }
    release(&lock);
}
```


Simulation of Readers/Writers Solution

W1 finishes (R3 still waiting)

AR = 0, WR = 1, AW = 0, WW = 0

```
Writer() {
    acquire(&lock);
    while ((AW + AR) > 0) {           // Is it safe to write?
        WW++;                         // No, Active users exist
        cond_wait(&okToWrite, &lock); // Sleep on cond var
        WW--;                         // No longer waiting
    }
    AW++;
    release(&lock);
}
```

AccessDBase(ReadWrite);

```
    acquire(&lock);
    AW--;
    if (WW > 0) {
        cond_signal(&okToWrite);
    } else if (WR > 0) {
        cond_broadcast(&okToRead);
    }
    release(&lock);
}
```

Simulation of Readers/Writers Solution

W1 signaling readers (R3 still waiting)

AR = 0, WR = 1, AW = 0, WW = 0

```
Writer() {
    acquire(&lock);
    while ((AW + AR) > 0) {           // Is it safe to write?
        WW++;                         // No, Active users exist
        cond_wait(&okToWrite, &lock); // Sleep on cond var
        WW--;                         // No longer waiting
    }
    AW++;
    release(&lock);
}
```

AccessDBase(ReadWrite);

```
    acquire(&lock);
    AW--;
    if (WW > 0) {
        cond_signal(&okToWrite);
    } else if (WR > 0) {
        cond_broadcast(&okToRead);
    }
    release(&lock);
}
```

Simulation of Readers/Writers Solution

R3 gets signal (no waiting threads)

AR = 0, WR = 1, AW = 0, WW = 0

```
Reader() {
    acquire(&lock);
    while ((AW + WW) > 0) { // Is it safe to read?
        WR++;              // No. Writers exist
        cond_wait(&okToRead, &lock); // Sleep on cond var
        WR--;              // No longer waiting
    }
    AR++;                  // Now we are active!
    release(&lock);

    AccessDBase(ReadOnly);

    acquire(&lock);
    AR--;
    if (AR == 0 && WW > 0)
        cond_signal(&okToWrite);
    release(&lock);
}
```

Simulation of Readers/Writers Solution

R3 gets signal (no waiting threads)

AR = 0, WR = 0, AW = 0, WW = 0

```
Reader() {
    acquire(&lock);
    while ((AW + WW) > 0) { // Is it safe to read?
        WR++;              // No. Writers exist
        cond_wait(&okToRead, &lock); // Sleep on cond var
        WR--;              // No longer waiting
    }
    AR++;                  // Now we are active!
    release(&lock);

    AccessDBase(ReadOnly);

    acquire(&lock);
    AR--;
    if (AR == 0 && WW > 0)
        cond_signal(&okToWrite);
    release(&lock);
}
```

Simulation of Readers/Writers Solution

R3 accessing dbase (no waiting threads)

AR = 1, WR = 0, AW = 0, WW = 0

```
Reader() {
    acquire(&lock);
    while ((AW + WW) > 0) { // Is it safe to read?
        WR++;              // No. Writers exist
        cond_wait(&okToRead, &lock); // Sleep on cond var
        WR--;              // No longer waiting
    }
    AR++;                  // Now we are active!
    release(&lock);
}
```

AccessDBase(ReadOnly);

```
    acquire(&lock);
    AR--;
    if (AR == 0 && WW > 0)
        cond_signal(&okToWrite);
    release(&lock);
}
```

Simulation of Readers/Writers Solution

R3 finishes (no waiting threads)

AR = 1, WR = 0, AW = 0, WW = 0

```
Reader() {
    acquire(&lock);
    while ((AW + WW) > 0) { // Is it safe to read?
        WR++;                // No. Writers exist
        cond_wait(&okToRead, &lock); // Sleep on cond var
        WR--;                // No longer waiting
    }
    AR++;                    // Now we are active!
    release(&lock);
}
```

AccessDBase(ReadOnly);

```
    acquire(&lock);
    AR--;
    if (AR == 0 && WW > 0)
        cond_signal(&okToWrite);
    release(&lock);
}
```

Simulation of Readers/Writers Solution

R3 finishes (no waiting threads)

AR = 0, WR = 0, AW = 0, WW = 0

```
Reader() {
    acquire(&lock);
    while ((AW + WW) > 0) { // Is it safe to read?
        WR++;              // No. Writers exist
        cond_wait(&okToRead, &lock); // Sleep on cond var
        WR--;              // No longer waiting
    }
    AR++;                  // Now we are active!
    release(&lock);

    AccessDbase(ReadOnly);

    acquire(&lock);
    AR--;
    if (AR == 0 && WW > 0)
        cond_signal(&okToWrite);
    release(&lock);
}
```

Questions

Can readers starve? Consider Reader() entry code:

```
while ((AW + WW) > 0) {    // Is it safe to read?
    WR++;                  // No. Writers exist
    cond_wait(&okToRead,&lock); // Sleep on cond var
    WR--;                  // No longer waiting
}
AR++;                      // Now we are active!
```

What if we erase the condition check in Reader exit?

```
AR--;                      // No longer active
if (AR == 0 && WW > 0)    // No other active readers
    cond_signal(&okToWrite); // Wake up one writer
```


Questions

Further, what if we turn the `signal()` into `broadcast()`

```
AR--;                                // No longer active  
cond_broadcast(&okToWrite); // Wake up sleepers
```

Finally, what if we use only one condition variable (call it “`okContinue`”) instead of two separate ones?

- Both readers and writers sleep on this variable
- Must use `broadcast()` instead of `signal()`

Code for a Reader

```
Reader() {
    // First check self into system
    acquire(&lock);
    while ((AW + WW) > 0) {    // Is it safe to read?
        WR++;                // No. Writers exist
        cond_wait(&okToRead,&lock); // Sleep on cond var
        WR--;                // No longer waiting
    }
    AR++;                    // Now we are active!
    release(&lock);

    // Perform actual read-only access
    AccessDatabase(ReadOnly);

    // Now, check out of system
    acquire(&lock);
    AR--;                    // No longer active
    if (AR == 0 && WW > 0)    // No other active readers
        cond_signal(&okToWrite); // Wake up one writer
    release(&lock);
}
```

Code for a Writer

```
Writer() {
    // First check self into system
    acquire(&lock);
    while ((AW + AR) > 0) {    // Is it safe to write?
        WW++;                // No. Active users exist
        cond_wait(&okToWrite,&lock);    // Sleep on cond var
        WW--;                // No longer waiting
    }
    AW++;                    // Now we are active!
    release(&lock);
    // Perform actual read/write access
    AccessDatabase(ReadWrite);
    // Now, check out of system
    acquire(&lock);
    AW--;                    // No longer active
    if (WW > 0){             // Give priority to writers
        cond_signal(&okToWrite); // Wake up one writer
    } else if (WR > 0) {     // Otherwise, wake reader
        cond_broadcast(&okToRead); // Wake all readers
    }
    release(&lock);
}
```

Mesa Monitor Conclusion

Monitors represent the synchronization logic of the program

- Wait if necessary
- Signal when change something so any waiting threads can proceed

```
lock
while (need to wait) {
    condvar.wait();
}
unlock
```



Check and/or update
state variables
Wait if necessary

```
do something so no need to wait
lock

condvar.signal();

unlock
```



Check and/or update
state variables

C Language Support for Synchronization

C language: Pretty straightforward synchronization

Just make sure you know *all* the code paths out of a critical section

```
int Rtn() {  
    acquire(&lock);  
    ...  
    if (exception) {  
        release(&lock);  
        return errReturnCode;  
    }  
    ...  
    release(&lock);  
    return OK;  
}
```

Concurrency and Synchronization in C

Harder with more locks

```
void Rtn() {  
    lock1.acquire();  
    ...  
    if (error) {  
        lock1.release();  
        return;  
    }  
    ...  
    lock2.acquire();  
    ...  
    if (error) {  
        lock2.release();  
        lock1.release();  
        return;  
    }  
    ...  
    lock2.release();  
    lock1.release();  
}
```

C++ Language Support for Synchronization

Languages with exceptions like C++

- Languages that support exceptions are problematic (easy to make a non-local exit without releasing lock)

```
void Rtn() {  
    lock.acquire();  
    ...  
    DoFoo();  
    ...  
    lock.release();  
}  
void DoFoo() {  
    ...  
    if (exception) throw errException;  
    ...  
}
```

- Notice that an exception in DoFoo() will exit without releasing the lock!

C++ Language Support for Synchronization (con't)

Must catch all exceptions in critical sections

- Catch exceptions, release lock, and re-throw exception:

```
void Rtn() {  
    lock.acquire();  
    try {  
        ...  
        DoFoo();  
        ...  
    } catch (...) {           // catch exception  
        lock.release();      // release lock  
        throw;               // re-throw the exception  
    }  
    lock.release();  
}  
void DoFoo() {  
    ...  
    if (exception) throw errException;  
    ...  
}
```


Much better: C++ Lock Guards

```
#include <mutex>
int global_i = 0;
std::mutex global_mutex;

void safe_increment() {
    std::lock_guard<std::mutex> lock(global_mutex);
    ...
    global_i++;
    // Mutex released when 'lock' goes out of scope
}
```

Python with Keyword

More versatile than we show here (can be used to close files, database connections, etc.)

```
lock = threading.Lock()
```

```
...
```

```
with lock: # Automatically calls acquire()
```

```
    some_var += 1
```

```
...
```

```
# release() called however we leave block
```

Java synchronized Keyword

Every Java object has an associated lock:

- Lock is acquired on entry and released on exit from a `synchronized` method
- Lock is properly released if exception occurs inside a `synchronized` method
- Mutex execution of synchronized methods (beware deadlock)

```
class Account {  
    private int balance;  
  
    // object constructor  
    public Account (int initialBalance) {  
        balance = initialBalance;  
    }  
    public synchronized int getBalance() {  
        return balance;  
    }  
    public synchronized void deposit(int amount) {  
        balance += amount;  
    }  
}
```

Java Support for Monitors

Along with a lock, every object has a single condition variable associated with it

To wait inside a synchronized method:

- `void wait();`
- `void wait(long timeout);`

To signal while in a synchronized method:

- `void notify();`
- `void notifyAll();`

Summary: Synchronization

Programs	Shared Programs
Higher-level API	Locks Semaphores Monitors Send/Receive
Hardware	Load/Store Disable Interrupts Test&Set Compare&Swap

Implement various higher-level synchronization primitives using atomic operations

Topic Breakdown

Virtualizing the CPU

Process Abstraction and API

Threads and Concurrency

Scheduling

Virtualizing Memory

Virtual Memory

Paging

Persistence

IO devices

File Systems

Distributed Systems

Challenges with distribution

Data Processing & Storage